

Kapittel 9

Filer og indekser

Læringsmål

- ☐ Kjenne til egenskaper ved relevante lagringsmedier.
- ☐ Forstå hvordan ulike lagringsmedier blir brukt i et databasesystem.
- ☐ Forstå viktige teknikker for lagring og bruk av data på disk, blant annet forskjellige filstrukturer, søketeknikker og indekser.
- ☐ Kunne bruke SQL for å definere indekser.

Lagringsmedier

Minne:
ustabilt/flyktig



Cache

RAM

Lager:
stabilt



SSD

Harddisk

Optiske medier

Magnetbånd

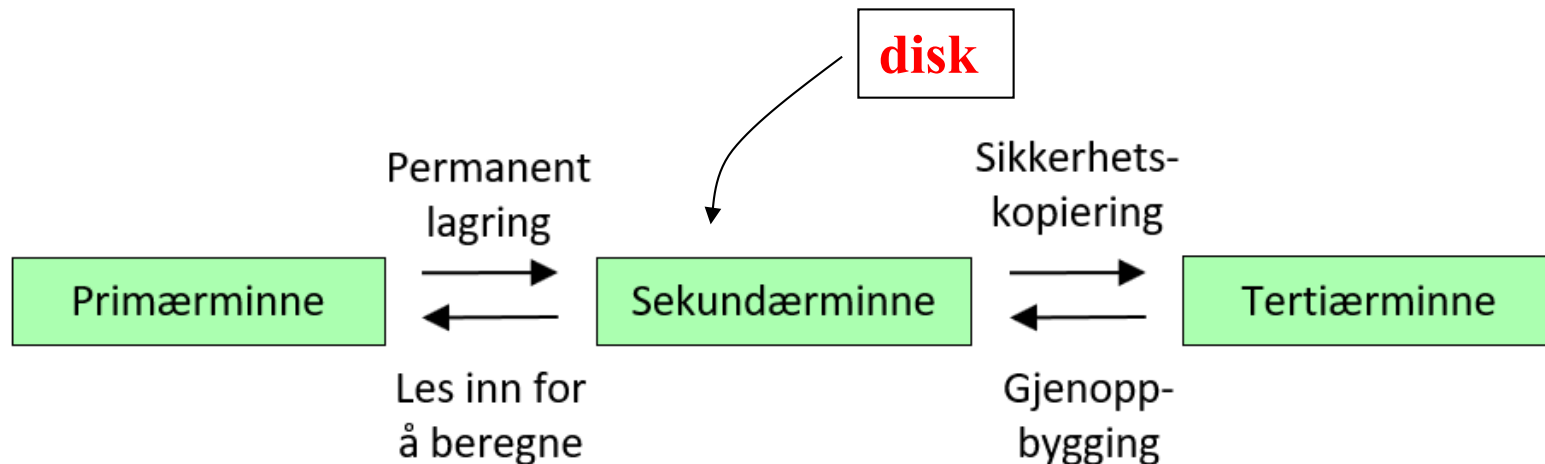
Hurtig, dyrt



Langsomt, billig

Bruk av lagringsmedier

- ❑ Minnet er (som regel) ikke stort nok til å holde hele databasen.
 - Data blir lest inn i minnet fra disk for å utføre beregninger (hurtig).
 - Oppdaterte data skrives tilbake til disk (stabilt lagringsmedium).
- ❑ Et DBHS skyfler data kontinuerlig mellom minne og disk.



Representasjon av tabeller

- ❑ I prinsippet kan vi tenke på ethvert (?) lagringsmedium som en **nummerert sekvens av byte**.
- ❑ Hvordan lagrer vi en databasetabell (2-dimensjonal) som en byte-sekvens på platelageret?
 - Rad-for-rad eller kolonne-for-kolonne?
 - Hver enkelt verdi krever gjerne flere byte.
 - Ett enkelt tegn kan kreve mer enn 1 byte (Unicode).
- ❑ En database lagret på ytre lager er organisert i en eller flere **filer**.
 - En fil består av **poster** som igjen består av **felt**.
- ❑ **Forenkling:**
 - Hver tabell lagres på en (egen) fil.
 - Hver rad lagres i en post, og hver verdi i et felt.

7
P
e
r
...
8
...

Terminologi

- ❑ Fra SQL til relasjonsmodellen til ER til fysiske filer:

SQL	Rel.mod.	ER	Filer
tabell	relasjon	entitetstype	fil
rad	tuppel	entitetsforekomst	post
kolonne	attributt	attributt	felt



- ❑ Merk: Begrepene betyr ikke helt det samme.

Blokker

- ❑ En harddisk er svært mye tregere enn minnet.
- ❑ En **blokk** er den minste enheten for overføring av data mellom minnet og ytre lager.
 - Typisk **blokkstørrelse**: 4KB (4 096 byte).
 - En blokk vil som regel inneholde **mange rader**.
- ❑ DBHS holder oversikten over hvilke data som ligger i hvilke blokker.

Fil delt inn i blokker

	KNr	Fornavn	Etternavn	Telefon	
Post 1	1	Elias	Hansen	99 88 77 66	} Blokk 1
Post 2	2	Hulda	Akselsen	31 45 88 21	
Post 3	5	John	Boine	23 94 53 18	
	...	...	...	...	
Post 58	63	Mari	Gygre	55 66 77 88	} Blokk 2
Post 59	64	Michael	Svensen	19 82 37 64	
Post 60	69	Robert	Romman	91 28 73 46	
Post 61	72	Laura	Eika	64 37 82 19	
	...	...	...	...	
Post 116	129	Andreas	Karlsen	51 15 83 38	
	...	...	...	...	

- ❑ En fil blir lagret i et antall blokker.
- ❑ Det settes gjerne av litt ledig plass i hver blokk.
 - Gjør det lettere å sette inn / slette rader (poster).
 - **Fyllingsgraden** (f.eks. 80%) kan styres i noen DBHS.

Filstrukturer

❑ **Ikke-sorterte filer** (Heap-filer): «Tilfeldig» rekkefølge.

➤ Nye poster legges inn til slutt.

87	27	7	16	41	19	50			
----	----	---	----	----	----	----	--	--	--

Hver boks symboliserer en post.

❑ **Sorterte filer** (sekvensielle filer): Sortert, f.eks. med hensyn på VNr.

7	16	19	27	41	50	87			
---	----	----	----	----	----	----	--	--	--

Et antall poster i hver blokk

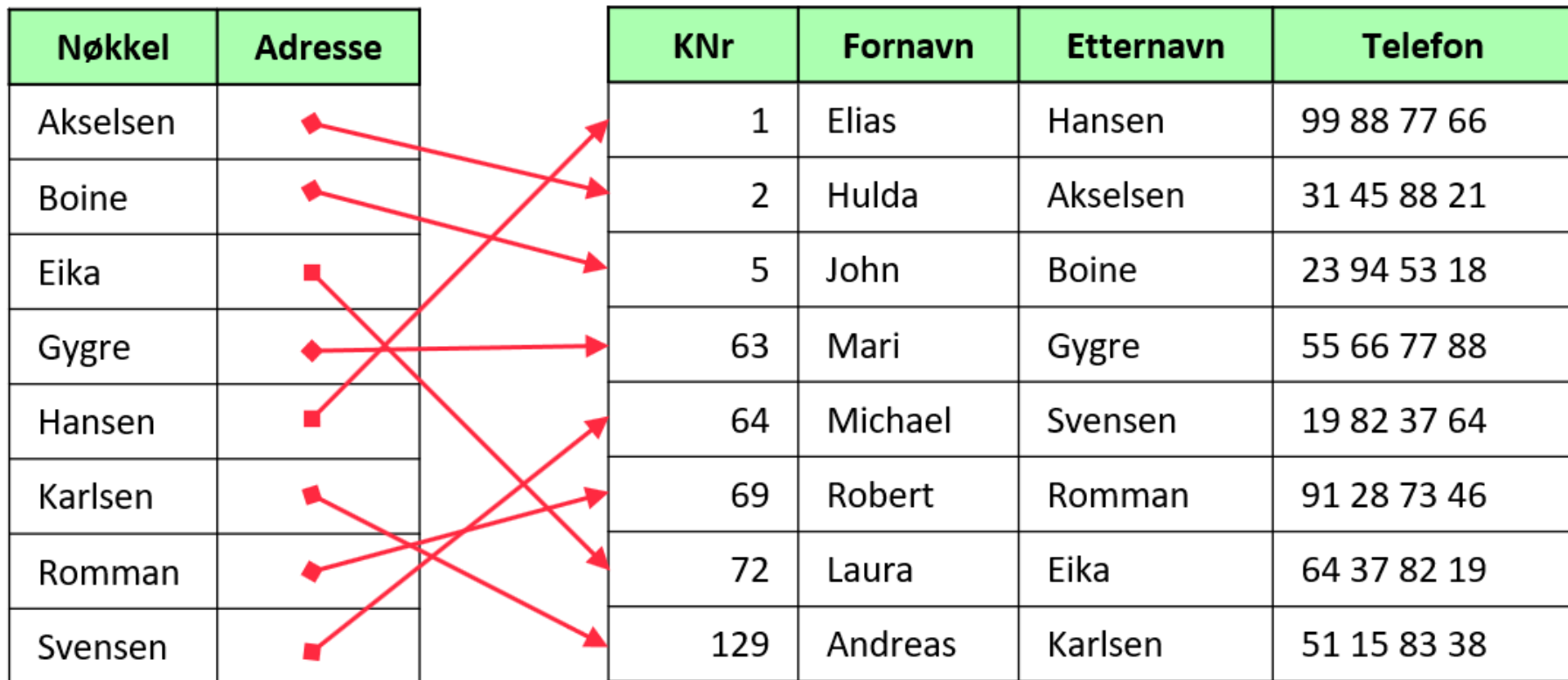
Indekser

- ❑ En **indeks** er en datastruktur som kan **effektivisere** søk.
 - Kan lage **flere indekser** på en tabell.
 - Men krever ekstra **lagringsplass**
 - Må holdes oppdatert (**merarbeid ved endringer**)

- ❑ Tenk på en indeks som en referanseliste bakerst i en bok.
 - Inneholder **søkeord + referanser** (=sidetall i en bok).
 - Listen er **sortert** med hensyn på søkeord.
 - I databaseindekser vil referansene være diskadresser.

- ❑ Det er raskere å søke i indeksen enn i datafilen, fordi:
 - Indeksen er **mindre** og den er **sortert**

Visualisering av indekser



Lage indekser med SQL

- ❑ Den enkleste varianten:

```
CREATE INDEX VarenavnIdx  
ON Vare ( Betegnelse )
```

- ❑ Indekser kan hindre repetisjoner:

```
CREATE UNIQUE INDEX EtternavnIdx  
ON Ansatt ( Etternavn )
```

- ❑ Indekser kan være sammensatte:

```
CREATE INDEX NavnIdx  
ON Ansatt ( Etternavn, Fornavn )
```

Får her indeks på etternavn «på kjøpet» !

Tette og ikke-tette indekser

- ❑ En **tett indeks** (engelsk: dense) har en post for hver eneste **post** i filen.
- ❑ En **ikke-tett indeks** (engelsk: sparse) har en post for hver **blokk** i filen.
- ❑ Ikke-tette indekser er mindre enn tette.
 - Som regel mange poster i hver blokk.
- ❑ Maks 1 ikke-tett indeks pr. datafil
 - En datafil kan kun være sortert på én måte, f.eks. på kundenr.
 - Bare indeksen på kundenr kan da være ikke-tett.

B⁺-trær

❑ Flernivå-indekser:

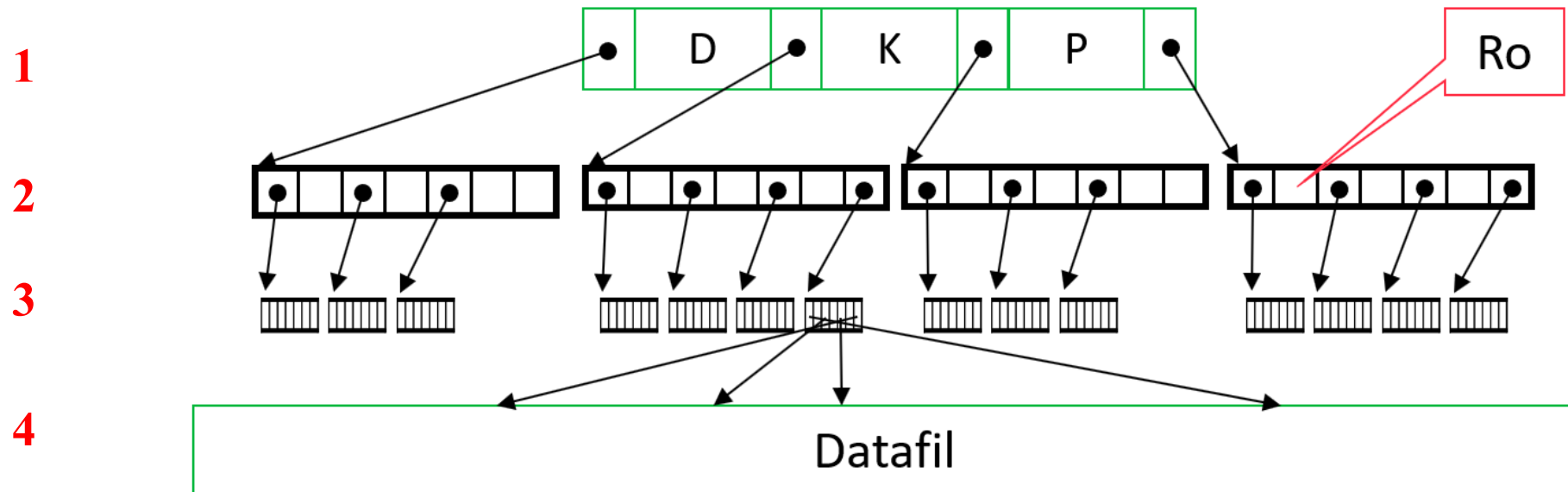
- For store indekser kan man lage «indekser-på-indekser».
- Indeksene danner da en **tre-struktur**.
- Indeksene på nivå 2, 3, 4 osv. kan være ikke-tette.
 - Nivået under er en indeks, som jo er sortert.
- Søking skjer fra «roten» i treet mot «bladene».
- Antall diskaksesser = **dybden i treet**.

❑ B⁺-trær er en vanlig form for (flernivå-)indeks:

- Like mange diskaksesser for alle søkeverdier: Treet er Balansert.

Antall lese-
operasjoner:

B⁺-trær



Anta vi skal finne post der Etternavn = «Roheim».

- Les inn rot-blokken.
- Søk i rot-blokken: Her skal vi følge pekeren helt til høyre.
- Gjenta søk på mellomliggende nivåer ...
- ... Les inn riktig blokk fra datafilen.

Valg av datatyper

❑ Tekst

- Bruk CHAR(n) hvis verdiene har en **fast lengde** (fødselsnr, regnr for biler), og VARCHAR(n) ellers.
- Sett stor nok maks-grense.
- Egne datatyper for svært lange tekster

❑ Tall

- Velg så «liten» datatype som mulig, men slik at alle tenkelige verdier får plass.
- Heltall eller desimaltall?
- Fast antall desimaler? Kronebeløp?

❑ Tekst eller tall?

- Skal du regne med verdiene?

❑ Tid

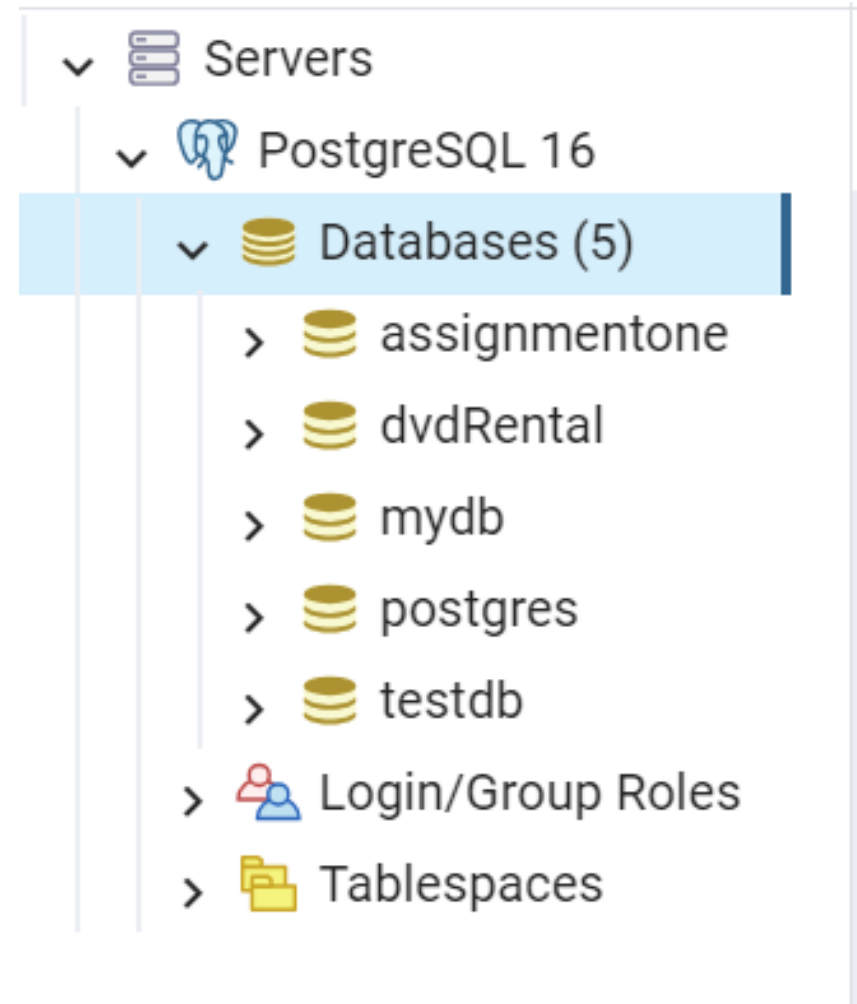
- Dato og/eller klokkeslett?

Valg av indekser

- ❑ Følgende kolonner **kan med fordel** indekseres:
 - Kolonner man ofte søker i, eller sorterer på.
 - Primærnøkler (som regel automatisk indeksert)
 - Fremmednøkler (kan også være automatisk indeksert)
 - Kolonner som ofte brukes som koblingskolonner i spørringer.
- ❑ Følgende kolonner **bør ikke/kan ikke** indekseres:
 - Kolonner som kun inneholder få ulike verdier/mange like verdier, for eksempel Ja/Nei-kolonner (bitmap-indekser mulig)
 - Bilder, video, lydklipp
 - Lange dokumenter (finnes spesielle teknikker, jmf søkemotorer)
- ❑ Indekser krever **maskinressurser**:
 - Indekser tar opp plass.
 - Indekser må vedlikeholdes ved oppdatering i tabellene.

PostgreSQL Storage Structure

- ❑ Database cluster is a collection of databases that is managed by a single instance of a running database server.
- ❑ Before you can do anything, you must initialize a database storage area on disk.
- ❑ After initialization, a database cluster will contain a database named “postgres”, which is meant as a default database for use by utilities, users and third-party applications.



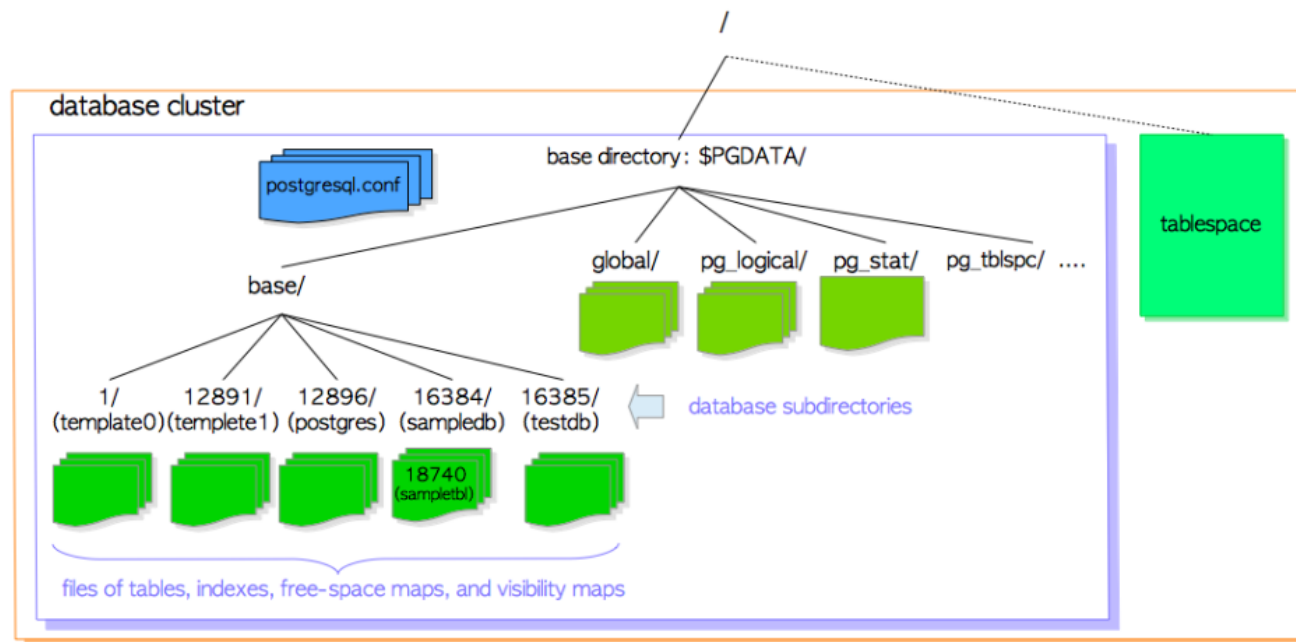
PostgreSQL Storage Structure

❑ Query the structure of “dvdRental” database:

Query					Query History				
1					SELECT relname, oid, relpages, reltuples FROM pg_class;				
Data Output					Messages				
	relname	oid	relpages	reltuples					
	name	[PK] oid	integer	real					
29	actor_pkey	16555	2	200					
30	address_pkey	16557	4	603					
31	category_pkey	16559	2	16					
32	address	16470	8	603					
33	category	16439	1	16					
34	city	16476	5	600					
35	country	16482	1	109					
36	customer	16423	9	599					
37	film_actor	16456	30	5462					
38	film_category	16460	6	1000					
39	inventory	16498	25	4581					
40	language	16504	1	6					
41	rental	16520	150	16044					

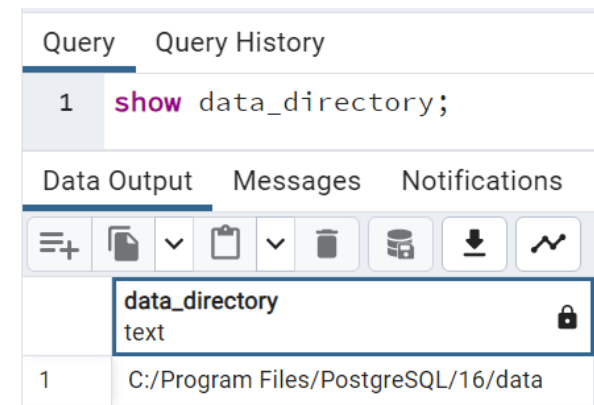
PostgreSQL Storage Structure

- ❑ In file system terms, the configuration and data files used by a database cluster are stored together within the cluster's data directory, commonly called PGDATA.
- ❑ The PGDATA directory contains several subdirectories and control files.
- ❑ The cluster configuration files postgresql.conf, pg_hba.conf, and pg_ident.conf are traditionally stored in PGDATA, although it is possible to place them elsewhere.

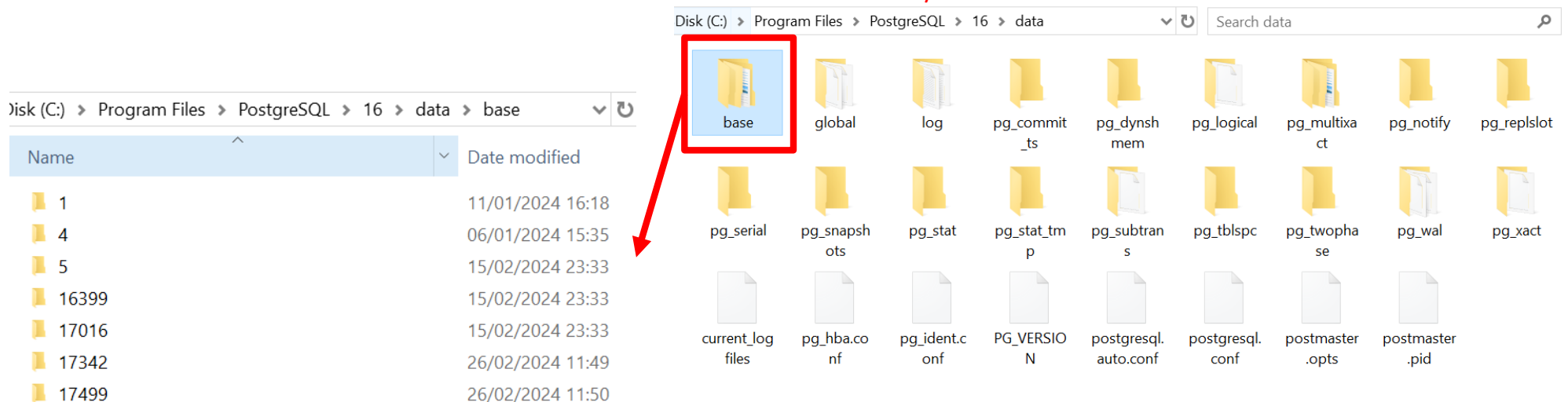


PostgreSQL Storage Structure

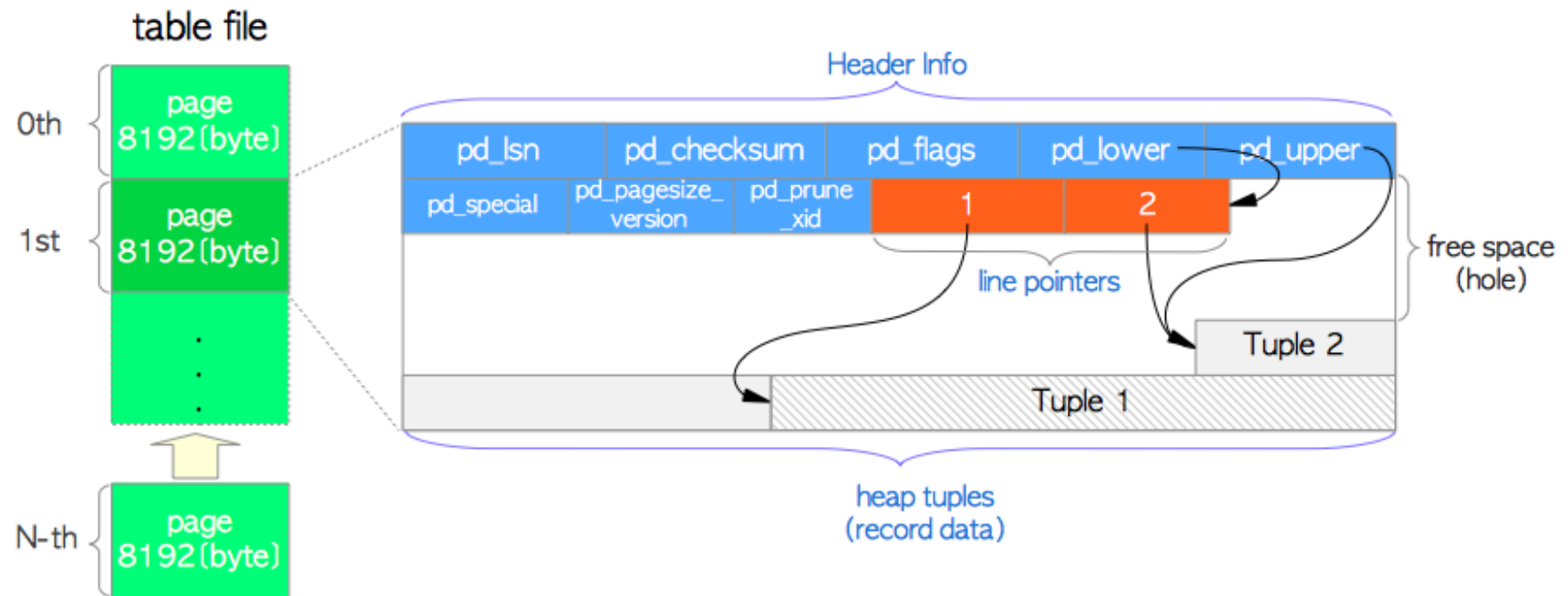
- ❑ For each database in the cluster there is a subdirectory within **PGDATA/base**, named after the database's OID in `pg_database`. This subdirectory is the default location for the database's files; in particular, its system catalogs are stored there.



PGDATA/base



PostgreSQL Storage Structure



PostgreSQL Storage Structure

- ❑ The OIDs listed in the query result represent the actual names for the databases in the physical file system. So, the database “dvdRental” is represented by a folder named “16399” and not “dvdRental”. So the path of “dvdRental” database can be: “C:\Program Files\PostgreSQL\16\data\base\16399”, where all files concerning this database are stored.

Disk (C:) > Program Files > PostgreSQL > 16 > data > base

Name	Date modified
1	11/01/2024 16:18
4	06/01/2024 15:35
5	15/02/2024 23:33
16399	15/02/2024 23:33
17016	15/02/2024 23:33
17342	26/02/2024 11:49
17499	26/02/2024 11:50

Query Query History

1 **SELECT** oid, datname **FROM** pg_database;

Data Output Messages Notifications

	oid [PK] oid	datname name
1	5	postgres
2	1	template1
3	4	template0
4	16399	dvdRental
5	17016	assignmentone
6	17342	testdb
7	17499	mydb

Query Query History

1 **SELECT** oid, datname **FROM** pg_database **WHERE** datname = 'dvdRental';

Data Output Messages Notifications

	oid [PK] oid	datname name
1	16399	dvdRental

You can also use this command for a specific database

PostgreSQL Storage Structure

- ❑ The same goes for the tables created in each database, created tables and objects are stored in the database's folder "16399" by their OID and not their textual name. PostgreSQL stores each table in the database in a separate file (a heap file), this file is named after the table's filenode number. This filenode number often matches the table's OID.

Program Files > PostgreSQL > 16 > data > base > 16399

Name	Date modified	Type
14842	15/01/2024 06:51	File
14842_fsm	15/01/2024 06:51	File
14842_vm	15/01/2024 06:51	File
14845	15/01/2024 06:45	File
14846	15/01/2024 06:51	File
16422	15/01/2024 06:55	File
16423	15/01/2024 06:55	File
16423_fsm	15/01/2024 06:55	File

What is fsm?

Query	Query History		
1	<code>select pg_relation_filepath('customer');</code>		
Data Output	Messages	Notifications	
			
	pg_relation_filepath text		The filenode number for «customer» table is 16423
1	base/16399/16423		

The filenode number for «customer» table is 16423

Query

Query History

1

SELECT pg_relation_filenode('customer');

Data Output

Messages

Notifications



	pg_relation_filenode oid	
1	16423	

To retrieve the filenode number for «customer» table without the file path

To retrieve the filenode number for «customer» table without the file path

Query	Query History	Or you could use this query
1		<code>SELECT oid FROM pg_class WHERE relname = 'customer';</code>

PostgreSQL Storage Structure

- ❑ Query the number of pages and tuples in table “rental” (the biggest table in “dvdRental” database:

Query Query History

1 SELECT relname, oid, reltuples, relpages FROM pg_class WHERE relname='rental';

Data Output Messages Notifications

What are the «relpages»?